



Save as PDF ▾



Upgrade



Wednesday 1:10 PM

in metamask, there a a token, I want to buy and it says not available in your region. how can I undrestand in what regions is able to buy?

Yes. The important thing is that **"not available in your region" in MetaMask usually refers to the buying/on-ramp provider**, not necessarily that the token itself is illegal or unavailable everywhere.

MetaMask says that its buy feature uses multiple providers, and **availability depends on your country/region, payment method, token, and network**. MetaMask automatically filters the available providers for your selected region. 🦊 MetaMask Help C... +1

How to find where the token can be bought

If you tell me the **exact token name or, even better, its contract address**, I can investigate it for you.

I can check:

1. **Which countries/regions are supported** for buying it.
2. Whether the restriction is from **MetaMask itself or a specific provider** such as MoonPay, Transak, Banxa, etc.
3. Whether the token can be bought through another exchange/provider.
4. Whether it is **actually tradable in Italy/EU**, if that's relevant to you.
5. Which **network** the token is on (Ethereum, BSC, Solana, Base, etc.).
6. Whether there is a direct way to buy it and then send it to MetaMask.

One important distinction

If MetaMask says:

"This token is not available in your region"

that's different from:

"There are no sellers/liquidity"

or

"This token is not supported."

For example, MetaMask itself says that if a quote appears in its list, the purchase is available under that quote; if no provider is available, you may need to acquire the token elsewhere and transfer it to MetaMask.  MetaMask Help C...

Send me the token name or its contract address (or a screenshot of the MetaMask message), and I'll check exactly which regions can buy it and whether there is a way for someone in Italy to buy it.



the token is zelion with symbol zln

Yes — I checked **Zelion (ZLN)** specifically.

What I found

The official ZelionTech website identifies **ZLN as a BEP-20 token on BNB Smart Chain**, with a maximum supply of 500 million and 0% transaction tax.  ZelionTech

I also found the ZLN contract listed on BNB Smart Chain as:

0x9d92d1fd041f687c48bb7d95de2edac541dc0181

So first, **make sure this is the ZLN you mean**. There are other tokens using the ZLN symbol.

Why MetaMask says it's unavailable

This is probably **not simply a country restriction on ZLN itself**.

MetaMask explains that its **Buy** function uses third-party on-ramp providers. Availability depends on your country, currency, payment method, and the particular token. MetaMask automatically shows you the providers available for your region.

 MetaMask Help C... +1

So there are two separate questions:

A. Can someone in Italy buy ZLN through MetaMask's Buy button?

VS.

B. Can someone in Italy acquire ZLN through another exchange/DEX and put it into MetaMask?

The fact that MetaMask says *"not available in your region"* only answers **A**. It doesn't necessarily mean **B** is impossible.

I can check the important part for you

I can now investigate **ZLN specifically** and find:

- 🌐 Which countries MetaMask/on-ramp providers allow it in
- 🇮🇹 Whether **Italy** is restricted
- 💱 Where ZLN is currently tradable
- 🔄 Whether there's a BNB Chain DEX route
- 💳 Whether you can buy BNB/USDT somewhere and swap it for ZLN
- ⚠️ Whether there are any restrictions or warnings associated with ZLN

If your goal is **"I am in Italy and want to buy ZLN"**, I can check the actual routes available to you rather than just the MetaMask Buy button.



Yes — now I get the style you're after: **simple, believable, two-word tech/crypto names**, with a subtle connection to **Naghme + Kristin**, rather than something that sounds artificially generated.

Here are some ideas:

◆ **Naghme + Kristin inspired**

- **Nagh Trust**
- **Nagh Core**
- **Nagh Link**
- **Nagh Labs**
- **Nagh Network**
- **Nagh Chain**
- **Nagh Works**
- **Nagh Systems**

Using **Kri/Kris**:

- **Kri Trust**
- **Kri Core**
- **Kri Link**
- **Kris Labs**

- Kris Network
- Kris Chain
- Kris Works
- Kris Systems

◆ More subtle combinations

These hide the two names more naturally:

- **Nakr Trust** — Nagh + K(ris)
- **Nakr Labs**
- **Nakr Link**
- **Nakr Core**
- **Nakr Network**
- **Nagris Labs**
- **Nagris Core**
- **Nagris Network**
- **Nagrin Labs**
- **Nagrin Systems**

★ My favorites

For something that you could realistically put on a website:

Nakr Labs

Blockchain infrastructure & digital assets

Nagrin Network

Decentralized digital infrastructure

Nakr Trust

Trusted digital transactions

Nagh Core

Digital assets & blockchain technology

Nakr Link

Connecting digital assets

I particularly like **Nakr Labs** for our experiment because it sounds like a legitimate small blockchain startup, while **Nakr** subtly combines the beginnings of **Naghme + Kristin**.

We could then have:

Nakr Labs
NKR — Nakr Token

That would look quite realistic in the demonstration without pretending to be Zelion.



ok, I choosed 'bella coin' with icon 'BLC' for the name.
let's do other steps now

Perfect. **Bella Coin (BLC)** it is.

We'll build this as a **technical demonstration**, not as an investment product or a copy of Zelion. We'll first reproduce the basic token stage safely on **BNB Smart Chain Testnet**, then later decide whether we even need mainnet.

Our target

Setting	Bella Coin
Name	Bella Coin
Symbol	BLC
Standard	BEP-20
Network	BNB Smart Chain Testnet initially
Total supply	500,000,000 BLC
Decimals	18
Minting	No additional minting
Tax	0%
Owner privileges	None after deployment, ideally
Wallet	MetaMask

Step 1 — Prepare your development environment

Since you're a developer, let's use **Hardhat + Solidity + OpenZeppelin**, rather than a token-generator website. This makes the demonstration much stronger because you can show your boss the actual source code and deployment process.

If you're currently on **Ubuntu**, open Terminal and run:

```
</> Bash   
  
mkdir bella-coin  
cd bella-coin  
  
npm init -y  
  
npm install --save-dev hardhat  
npm install @openzeppelin/contracts
```

Then check:

```
</> Bash   
  
node --version  
npm --version
```

I'd like to know those two versions before we continue, because Hardhat's current setup has changed across versions.

Don't deploy anything yet

For now, **don't send any real BNB and don't use mainnet**.

Once you send me the output of:

```
</> Bash   
  
node --version  
npm --version
```

I'll give you **Step 2: create the Bella Coin smart contract**, and we'll build the exact 500-million BLC fixed-supply token.



here is the output, please also explain me step by stem
what we are doing. because I want to learn while doing

this. so I can expand my knowledge and experience.

→ bella-coin npm init -y

Wrote to /Users/imtech/Workspace/bella-coin/package.json:

```
{
  "name": "bella-coin",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC"
}
```

→ bella-coin npm install --save-dev hardhat

added 52 packages, and audited 53 packages in 9s

17 packages are looking for funding

run npm fund for details

found 0 vulnerabilities

→ bella-coin npm install @openzeppelin/contracts

added 2 packages, and audited 55 packages in 1s

17 packages are looking for funding

run npm fund for details

found 0 vulnerabilities

→ bella-coin node --version

v22.23.2

→ bella-coin npm --version

10.9.8

Show less ^

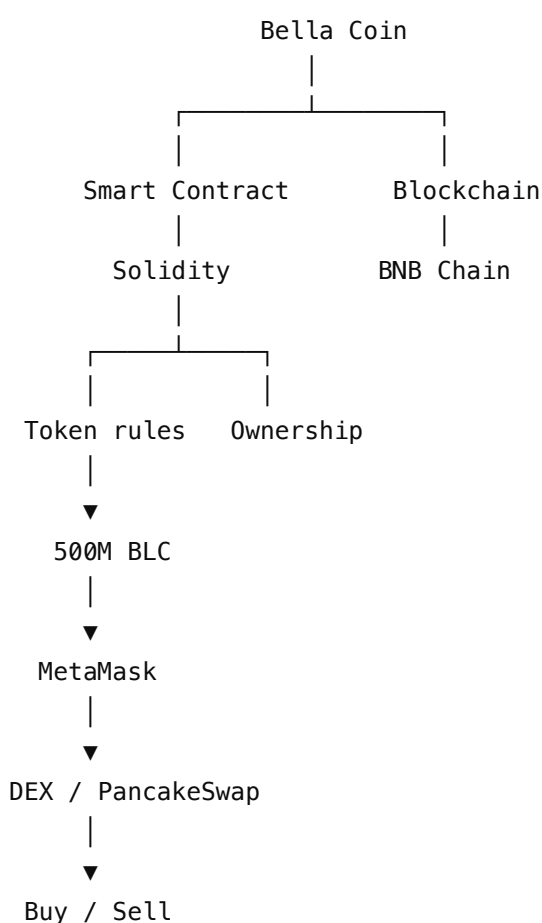
Perfect. Your environment is fine:

- **Node.js:** v22.23.2
- **npm:** 10.9.8
- **Hardhat:** installed
- **OpenZeppelin Contracts:** installed
- **No vulnerabilities reported**

And yes — let's do this as a **learning project**, not just a sequence of commands. I'll explain what every component does and, importantly, **why we're doing it**.

What we're building

Think of the project as several layers:



We're going to build these layers one at a time.

Step 2 — Create the Hardhat project

First: what is Hardhat?

Hardhat is a development environment for Ethereum-compatible blockchains.

BNB Smart Chain is EVM-compatible, so the same general Solidity tooling works there.

Hardhat helps us:

- compile Solidity
- test smart contracts
- deploy contracts
- interact with contracts
- run a local blockchain
- debug errors

So you can think of it roughly as:

Hardhat = development environment for blockchain smart contracts

It isn't the blockchain itself.

2.1 Initialize Hardhat

From your existing `bella-coin` directory, run:

```
</> Bash  
  
npx hardhat --init
```



You should get an interactive setup.

Depending on the Hardhat version, you'll see options for creating a project.

Choose the option for a **TypeScript/JavaScript Hardhat project** rather than an empty project if it asks.

If it asks about installing additional dependencies, allow it.

Why are we doing this?

You currently have:

```
bella-coin/  
├─ package.json  
├─ node_modules/  
└─ package-lock.json
```



After initializing Hardhat, we'll have something more like:

```
bella-coin/  
├─ contracts/  
└─ test/
```



```
|— ignition/  
|— scripts/  
|— node_modules/  
|— package.json  
|— hardhat.config...
```

The important directory initially is:

contracts/



That's where our Solidity smart contract will live.

Step 3 — Before writing the contract, understand what we're actually creating

This is an important concept.

When we say:

"We're creating Bella Coin."

we're **not creating a new cryptocurrency network**.

We're creating a **smart contract** on an existing blockchain.

Think of BNB Smart Chain as the operating environment.

Our contract will basically tell the blockchain:

There exists a token called Bella Coin, symbol BLC, with a supply of 500,000,000 units.

The blockchain stores the contract and its state.

Step 4 — What is BEP-20?

You will hear this term a lot.

BEP-20 is a token standard used on BNB Smart Chain.

It's conceptually very similar to Ethereum's ERC-20 standard.

A token contract needs functionality such as:

```
balanceOf()  
transfer()  
approve()  
transferFrom()  
allowance()  
totalSupply()
```



For example:

```
Alice  
100 BLC  
  
    transfer 20 BLC  
      ↓  
  
Bob  
20 BLC
```



The smart contract records the balances.

Step 5 — We're NOT going to write all of that ourselves

This is an important professional practice.

OpenZeppelin provides well-tested implementations of common token standards.

We already installed:

```
</> Bash  
  
npm install @openzeppelin/contracts
```



So instead of writing hundreds of lines of token-accounting logic ourselves, we'll use OpenZeppelin's ERC-20 implementation.

Conceptually:

```
</> solidity  
  
OpenZeppelin ERC20  
    ↓  
  BellaCoin  
    ↓  
    BLC
```



Our contract will mainly tell OpenZeppelin:

Create an ERC-20 token called Bella Coin, symbol BLC, and initially create 500 million tokens.

This is one reason creating a basic token isn't particularly mysterious or difficult for a competent developer.

Step 6 — Create the contract

After you've completed:

```
</> Bash
```

```
npx hardhat --init
```



look inside the project.

You should have a `contracts` directory.

Create:

```
contracts/BellaCoin.sol
```



Put this inside:

```
</> solidity
```

```
// SPDX-License-Identifier: MIT
```

```
pragma solidity ^0.8.20;
```

```
import {ERC20} from "@openzeppelin/contracts/token/ERC20/ERC20.sol";
```

```
contract BellaCoin is ERC20 {
```

```
    constructor() ERC20("Bella Coin", "BLC") {
```

```
        _mint(msg.sender, 500_000_000 * 10 ** decimals());
```

```
    }
```

```
}
```



Now let's understand **every part**.

pragma

```
</> solidity
```



```
pragma solidity ^0.8.20;
```

This tells the compiler which Solidity versions are compatible with our contract.

We're saying:

Use Solidity 0.8.20 or a compatible later 0.8.x version.

Import

```
</> solidity  
  
import {ERC20} from "@openzeppelin/contracts/token/ERC20/ERC20.sol";
```

We're importing OpenZeppelin's ERC-20 implementation.

This gives our contract the standard token functionality.

Without it, we'd have to implement things like:

```
balances  
transfer  
approve  
allowance  
transferFrom  
totalSupply
```

ourselves.

Contract declaration

```
</> solidity  
  
contract BellaCoin is ERC20 {
```

This means:

Create a smart contract called BellaCoin that inherits the functionality of ERC20.

So BellaCoin gets all the standard ERC-20 functionality.

Constructor

```
</> solidity  
  
constructor() ERC20("Bella Coin", "BLC") {
```



The constructor executes **once**, when the contract is deployed.

We're giving the underlying ERC-20 contract:

```
Name:   Bella Coin  
Symbol: BLC
```



After deployment, these become properties of the token.

The most important line

```
</> solidity  
  
_mint(msg.sender, 500_000_000 * 10 ** decimals());
```



This creates the initial supply.

We're saying:

Mint 500 million BLC and give all of them to the wallet that deploys the contract.

`msg.sender` means:

The address that is currently calling/deploying the contract.

So if your MetaMask address is:

```
0xABC...
```



then after deployment:

```
0xABC...  
|  
└─ 500,000,000 BLC
```



There is **no additional mint function** in our contract.

Therefore, after deployment, the total supply remains fixed at:

500,000,000 BLC

Why 10 ** decimals() ?

This is another important concept.

ERC-20 tokens normally use:

18 decimals



So internally:

1 BLC



is represented as:

1,000,000,000,000,000,000



smallest units.

Therefore:

```
</> solidity
```



```
500_000_000 * 10 ** 18
```

is the actual integer amount stored internally.

You don't normally see those 18 decimal places in MetaMask because the wallet displays the human-readable value.

One important correction from our earlier plan

We're on **macOS**, not Ubuntu, so I'll tailor the commands to your environment.

Also, we're going to do this **on testnet first**.

The progression will be:

```
Mac
```

```
|
```

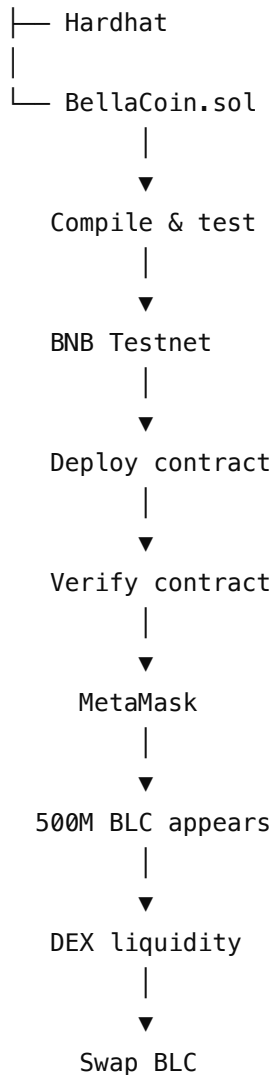
```
|
```

```
|
```

```
|
```

```
├─ Solidity
```





Only after we understand all of that would we consider whether a **mainnet deployment** is appropriate. There is no reason to spend real money while we're learning.

Your next action

Don't do anything else yet.

Run:

`</> Bash`

`npx hardhat --init`



Then **paste the complete output here**.

I'll explain what each option means and tell you exactly which one to choose. After that we'll compile Bella Coin and, before deploying it anywhere, I'll show you how to **test the contract locally**.

That way you're not just following a tutorial — by the end you'll understand what a smart contract, token standard, deployment, wallet, gas, testnet, liquidity pool, and DEX actually are.



ChatGPT can make mistakes. Check important info.